

编译原理

6. Activation Record, Cont.

rainoftime.github.io

浙江大学

计算机科学与技术学院

课程内容

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
5. Semantic Analysis
- 6. Activation Record**
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

Outline

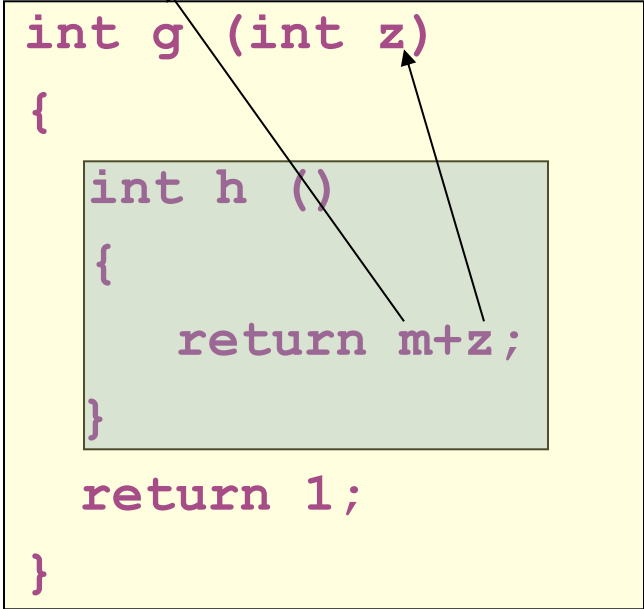
- 1 **Stack Frame**
- 2 **Use of Registers**
- 3 **Frame-Resident Variables**
- 4 **Block Structure**
- 5 **Block Structure: Imple.**
- 6 **A Typical Layout for Tiger**

4. Block Structure

Motivation: Implementing Block Structure

- **Block Structure:** In languages allowing nested function declarations (such as Tiger), the inner functions may use variables declared in outer functions.
- We can access local variables through the **Frame Pointer** (notice, the actual value of FP is unknown until runtime, but the each local-variable's *offset to FP* is known at compile time!)
- **Problem:** How can **h** access the “non-local” variables **m**?

```
int f (int x, int y)
{
    int m;
    int g (int z)
    {
        int h ()
        {
            return m+z;
        }
        return 1;
    }
    return 0;
}
```



5. Block Structure: Implementation

- **Static Link**
- **Display**
- **Lambda Lifting**

Strategies for Implementing Block Structure

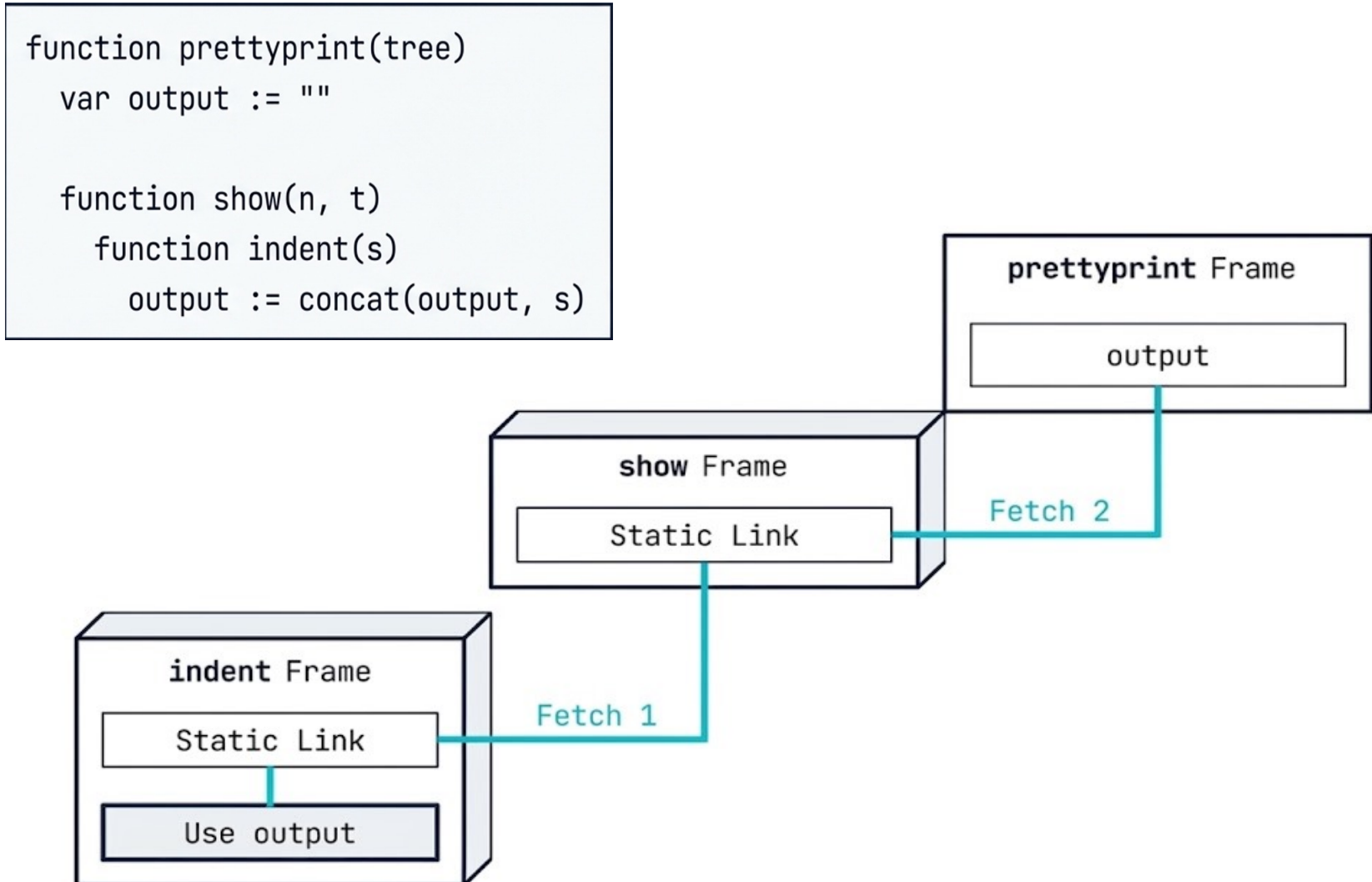
- **Static link (重点)**
 - Whenever a function g is called, it can be passed a pointer to the frame of the function statically enclosing
- **Display (可能重点)**
 - A *global array* can be maintained, containing – in position i – a pointer to the frame of the most recently entered procedure whose static nesting depth is i .
- **Lambda Lifting**
 - When f calls g , each variable of f that is actually accessed by g (or by any function nested inside g) is passed to g as an extra argument.

I: Static Link

- **The static link:** Whenever g is called, it is passed pointer to most recent activation record of f that immediately encloses g in program text
- **Using static links to access non-local data**
 - Each function is annotated with its enclosing depth
 - When a function at depth n accesses a variable at depth m : Emit code to climb up $n-m$ links to visit the appropriate activation record

The static link is a pointer to the activation record of the enclosing procedure

Example: Static Links (A High-Level Overview)



Example: Static Links

```
function main() =  
  let var global := 100  
  function level2(x: int) =  
    let var a := x + global  
    function level3(y: int) =  
      let var b := y + a  
      function level4(z: int) =  
        print(z + b + a + global)  
      in level4(y + 1) end  
    in level3(a + 1) end  
  in level2(10) end
```

print(z + b + a + global) in level4:

- **z**: Local variable, direct access within level4's frame
- **b**: Follow static link 1 time (to level3's frame) to access b
- **a**: Follow static link 2 times (to level3, then to level2's frame) to access a
- **global**: Follow static link 3 times (to level3, level2, then to main's frame) to access global

Variable	Defined in Level	Current Level	Links to Follow	Access Path
z	level4 (level 4)	level4 (level 4)	0	Direct access
b	level3 (level 3)	level4 (level 4)	1	Static Link → level3's frame → b
a	level2 (level 2)	level4 (level 4)	2	Static Link → level3's frame → Static Link → level2's frame → a
global	main (level 1)	level4 (level 4)	3	Static Link → level3's frame → Static Link → level2's frame → Static Link → main's frame → global

Example: Code Generation in Static Links (供参考)

```
function main() =  
  let var global := 100  
  function level2(x: int) =  
    let var a := x + global  
    function level3(y: int) =  
      print(y + a + global)  
    in level3(a + 1) end  
  in level2(10) end
```

典型的x86-64栈帧布局

[rbp+16]	函数参数 (如: y值)
[rbp+8]	静态链接 (指向词法父函数的帧)
[rbp]	保存的上一个rbp值
[rbp-8]	局部变量
[rbp-16]	更多局部变量...

函数调用设置

; 从level2调用level3:

```
push 111 ; 将y+1参数压入栈中  
push rbp ; 将当前帧指针作为静态链接压入栈中  
call level3  
add rsp, 16 ; 调整栈指针(8字节参数+8字节静态链接)
```

访问非局部变量

; 在level3内部, 访问level2中的变量'a' (向上1级)

```
mov r10, [rbp+8] ; 从当前帧加载静态链接
```

```
mov rax, [r10-8] ; 从level2的帧中加载'a'的值 ;
```

; 在level3内部访问main中的变量'global' (向上2级)

```
mov r10, [rbp+8] ; 从当前帧加载静态链接 (指向level2)
```

```
mov r10, [r10+8] ; 加载level2的静态链接(指向main)
```

```
mov rbx, [r10-8] ; 从main的帧中加载'global'的值
```

Pros and Cons of Static Links

- **Pros**

- Little extra overhead on parameter passing (Each activation record only needs one additional pointer)

- **Cons**

- $O(n)$ time to access a variable n levels up
(Multiple memory accesses needed for deeply nested variables)

Static Link vs. Dynamic Link

- **Dynamic link** (also named “control link”) to the caller's activation record
- **Static link** to the lexical parent's activation record

	静态链接 (Static Link)	动态链接 (Dynamic Link)
用途	访问非局部变量（词法作用域）	函数返回和栈帧管理
指向	词法父函数的栈帧	调用者的栈帧
由谁决定	函数的词法结构（源代码）	运行时的调用序列
调用时传递	调用者显式传递	自动通过push rbp创建

注: 在x86-64架构中, 动态链接就是函数序言中保存的旧rbp值（push rbp后存储在栈上的那个值）。而静态链接通常存储在[rbp+8]位置。

5. Block Structure: Implementation

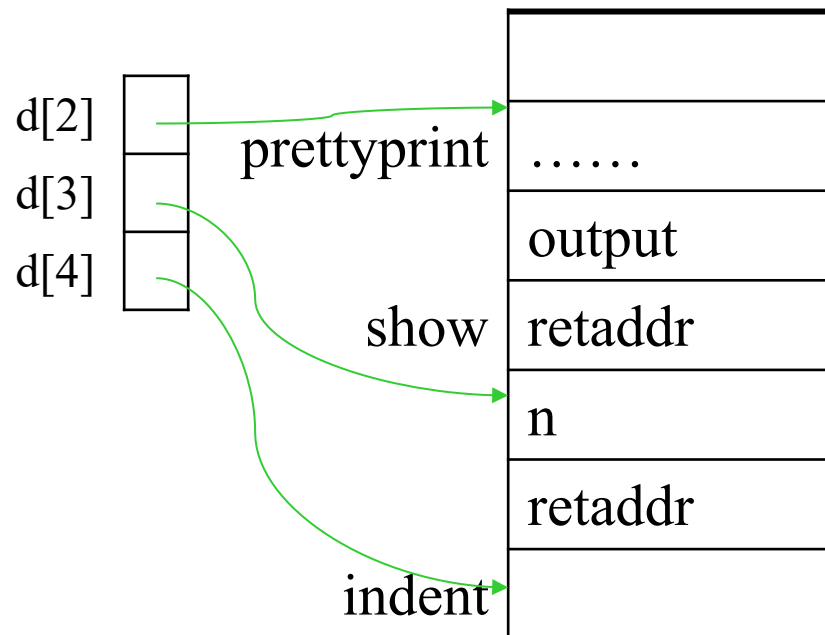
- **Static Link**
- **Display**
- **Lambda Lifting**

II: Display

- **Display**: a global array of pointers to frames
 - A global array containing frame pointers at each lexical level
 - $\text{Array}[i]$ = pointer to **most recent activation** at **level i**
 - Update display when entering/exiting a function
 - Direct access to any level (no chain traversal)

main	1
prettyprint	2
write	3
show	3
indent	4

Static nesting depth



Example: Display

```
function main() =  
  let var global := 100  
  function level2(x: int) =  
    let var a := x + global  
    function level3(y: int) =  
      let var b := y + a  
      function level4(z: int) =  
        print(z + b + a + global)  
      in level4(y + 1) end  
    in level3(a + 1) end  
  in level2(10) end
```

Display Array Structure

Display[1]	Pointer to most recent main (level 1) activation frame
Display[2]	Pointer to most recent level2 (level 2) activation frame
Display[3]	Pointer to most recent level3 (level 3) activation frame
Display[4]	Pointer to most recent level4 (level 4) activation frame

In level4, `print(z + b + a + global)`, accessing variables using Display:

- **z**: Local variable, directly accessible
- **b**: Display[3]→b (access level3's frame via Display[3])
- **a**: Display[2]→a (access level2's frame via Display[2])
- **global**: Display[1]→global (access main's frame via Display[1])

Example: Code Generation in Display (供参考)

```
function main() =  
  let var global := 100  
  function level2(x: int) =  
    let var a := x + global  
    function level3(y: int) =  
      print(y + a + global)  
    in level3(a + 1) end  
  in level2(10) end
```

使用Display Array访问非局部变量

; 在level3内部, 访问level2中的变量'a'(深度2)

mov r10, [display+16]; 获取level2的帧指针

mov rax, [r10-8]; 从level2的帧中加载'a'

这里display+16表示Display[2] (每个指针8字节, 所以索引2对应偏移16)

; 在level3内部, 访问main中的变量'global'(深度1)

mov r10, [display+8]; 获取level1(main)的帧指针

mov rcx, [r10-8]; 从main的帧中加载'global'

注: 也可以用一组特殊的寄存器来实现display array

Display Array的更新(以level2函数为例)

.data ; 数据段中的全局display数组

display: dq 0, 0, 0, 0; 4个嵌套级别

; 进入level2函数(深度2)

.text level2:

; 序言

push rbp

mov rbp, rsp

sub rsp, 16; 为局部变量分配空间

; 保存旧的display[2]并更新它

mov r10, [display+16]; 保存旧的display[2]值

push r10; 将其存储在栈上

mov [display+16], rbp; display[2] = 当前帧指针

; (略去) 函数体、调用level3函数、etc.

; 结语

pop r10; 恢复旧的display[2]

mov [display+16], r10

mov rsp, rbp

pop rbp; 恢复旧的rbp的值

ret

Pros and Cons

- **Advantages**

- *Efficient variable access*: $O(1)$ access to any lexical level (vs. $O(n)$ for static chain traversal)
- *Simpler code generation*: No need to follow multiple links for deep nesting

- **Disadvantages**

- *Complex context switching*: Need to save/restore Display entries on function entry/exit
- *Global state*: Display array is a global resource

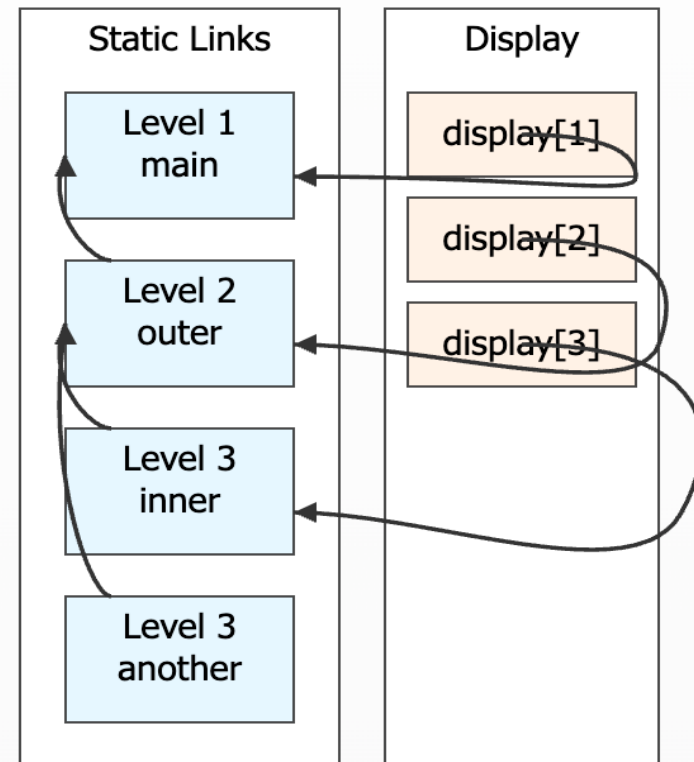
Static Link vs. Display

1. Static Links

- Each function frame gets a pointer to the frame of its **static parent**
- When calling a function, **pass pointer to correct enclosing frame**
- May require following a chain of links to access variables in outer scopes

2. Display

- A global array containing frame pointers at each lexical level. $\text{Array}[i] = \text{pointer to most recent activation at level } i$
- Update the display when entering/exiting a function
- Direct access to any level (no chain)



5. Block Structure: Implementation

- **Static Link**
- **Display**
- **Lambda Lifting**

III: Lambda Lifting

1. For each function, identify all **non-local variables** it uses
2. Add those variables as **extra parameters** to the function
3. Update all calls to pass the required variables

When **g** calls **f**, each variable of **g** that is actually accessed by **f** (or by any function *nested inside* **f**) is passed to **f** as an extra argument.

Example: Lambda Lifting

Rewrite the program by treating non-local variables as formal parameter

```
function main() =  
  let var global := 100  
    function level2(x: int) =  
      let var a := x + global  
        function level3(y: int) =  
          let var b := y + a  
            function level4(z: int) =  
              print(z + b + a + global)  
            in level4(y + 1) end  
          in level3(a + 1) end  
        in level2(10) end
```

```
function level4(z: int, b: int, a: int, global: int) =  
  print(z + b + a + global)
```

```
function level3(y: int, a: int, global: int) =  
  let var b := y + a  
  in level4(y + 1, b, a, global) end
```

```
function level2(x: int, global: int) =  
  let var a := x + global  
  in level3(a + 1, a, global) end
```

```
function main() =  
  let var global := 100  
  in level2(10, global) end
```

- No nested functions - all are at the same level
- No need for static links or display
- All variable access is to locals or parameters
- Functions explicitly pass all needed context to called functions

Pros and Cons

- **Advantages**

- *Simplified runtime environment*: No need to maintain complex stack frames with static links or displays.
- *Optimization opportunities*: Makes data flow explicit through parameters, etc.

- **Disadvantages**

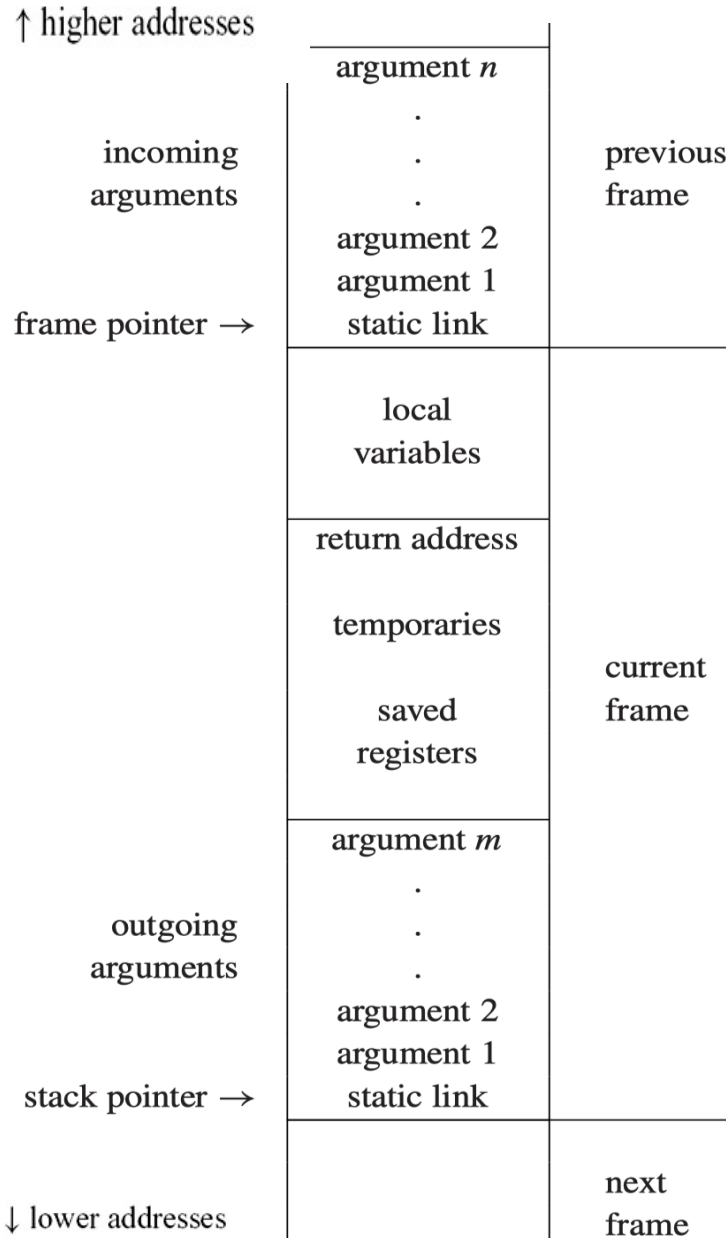
- *Performance*: Passing multiple extra parameters increases call overhead (**unnecessary passing when the variables aren't used**)
- *Limited applicability*: Not always suitable for higher-order functions

Summary

Feature	Static Links	Display	Lambda Lifting
Access to Non-local Variables	$O(n)$ - Follow chain	$O(1)$ - Direct access	$O(1)$ - Direct access (as parameters)
Space Requirements	One pointer per frame	Global array + one saved pointer per frame	Additional parameters for each function
Function Call/Return Overhead	Medium – Set one link	High - Save/restore display entries	High – Pass many parameters

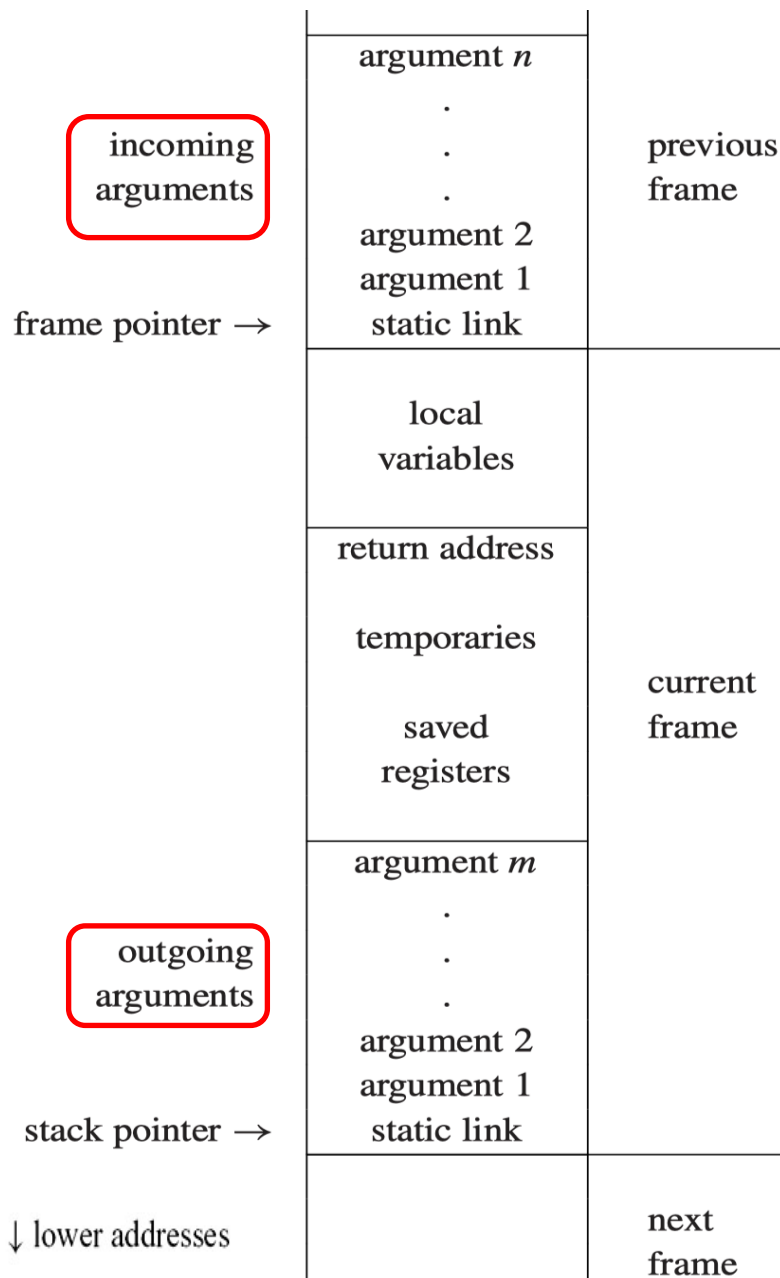
6. A Typical Stack Frame Layout for Tiger

A Typical Stack Frame Layout for Tiger



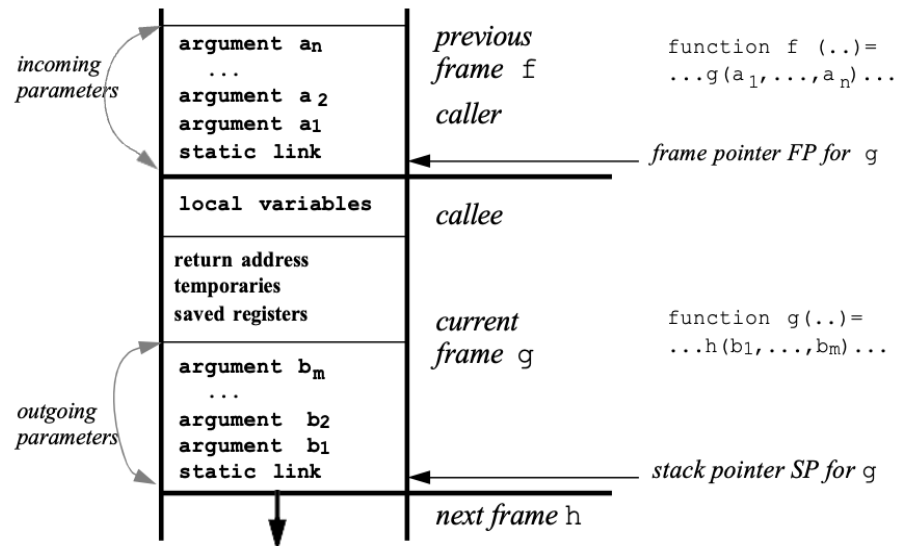
- **incoming arguments**: passed by the caller
- **return address**: where (within the calling function) control should return:
 - created by the CALL instruction
- some **local variables** and **temporaries** are in this frame, other local variables are kept in registers
- **saved registers**: make room for other uses of the registers
- **outgoing argument**: pass parameters to other functions
- **static link**

A Typical Stack Frame: Parameters

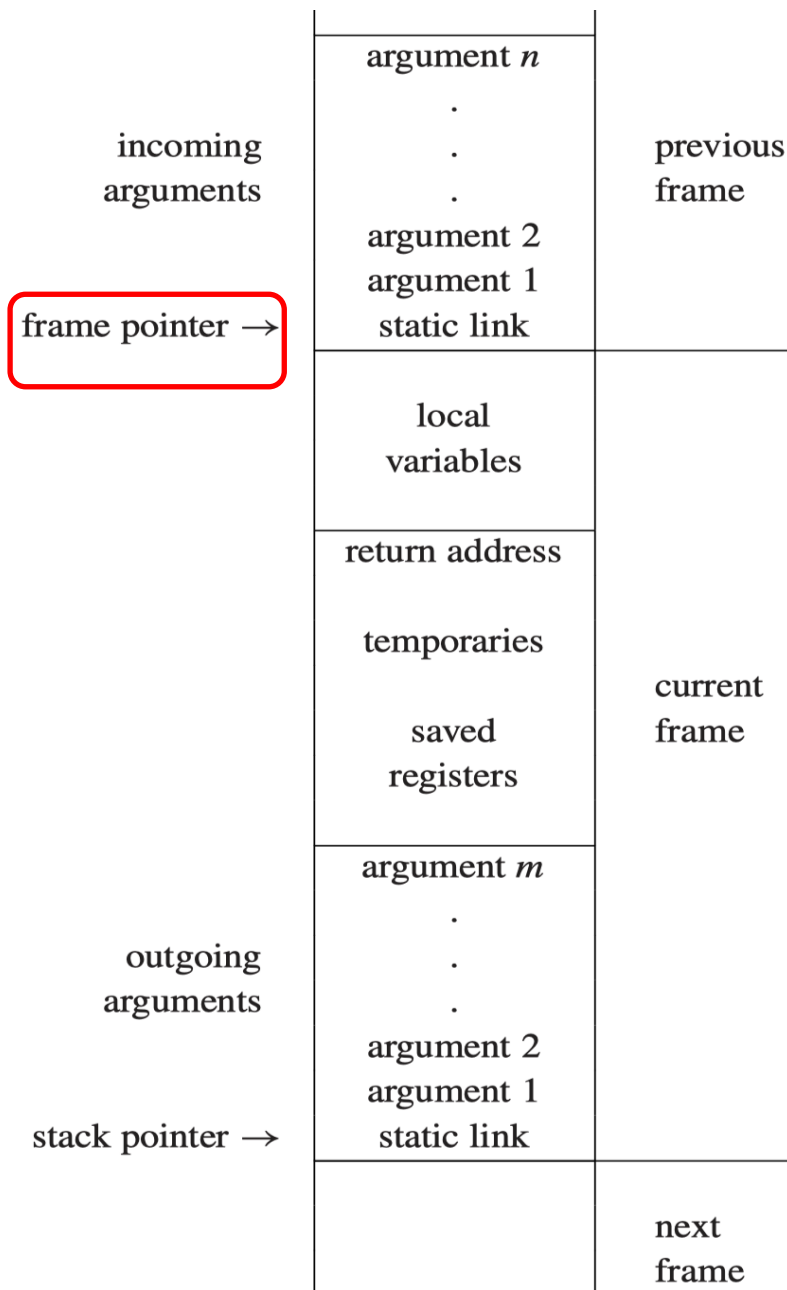


- **Incoming arguments:** passed by the caller
- **Outgoing argument:** pass parameters to other functions

一个具体的例子

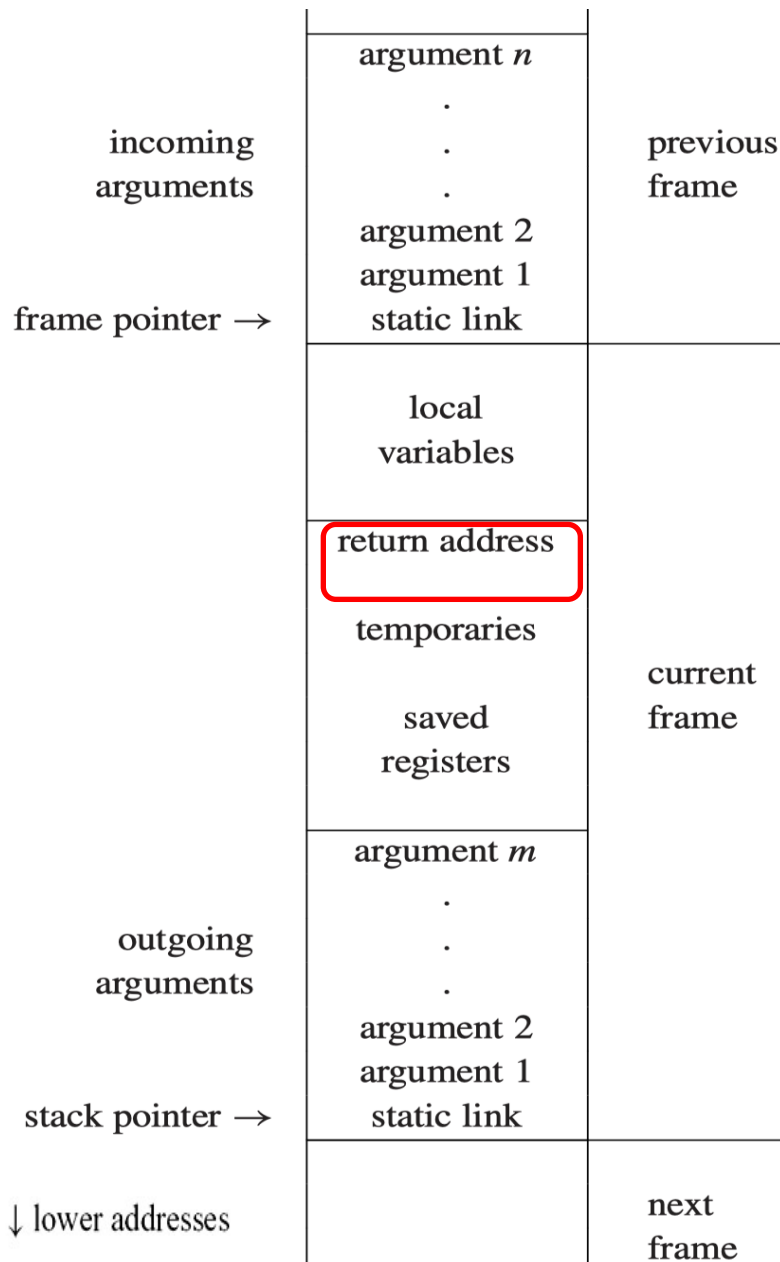


A Typical Stack Frame: Frame Point



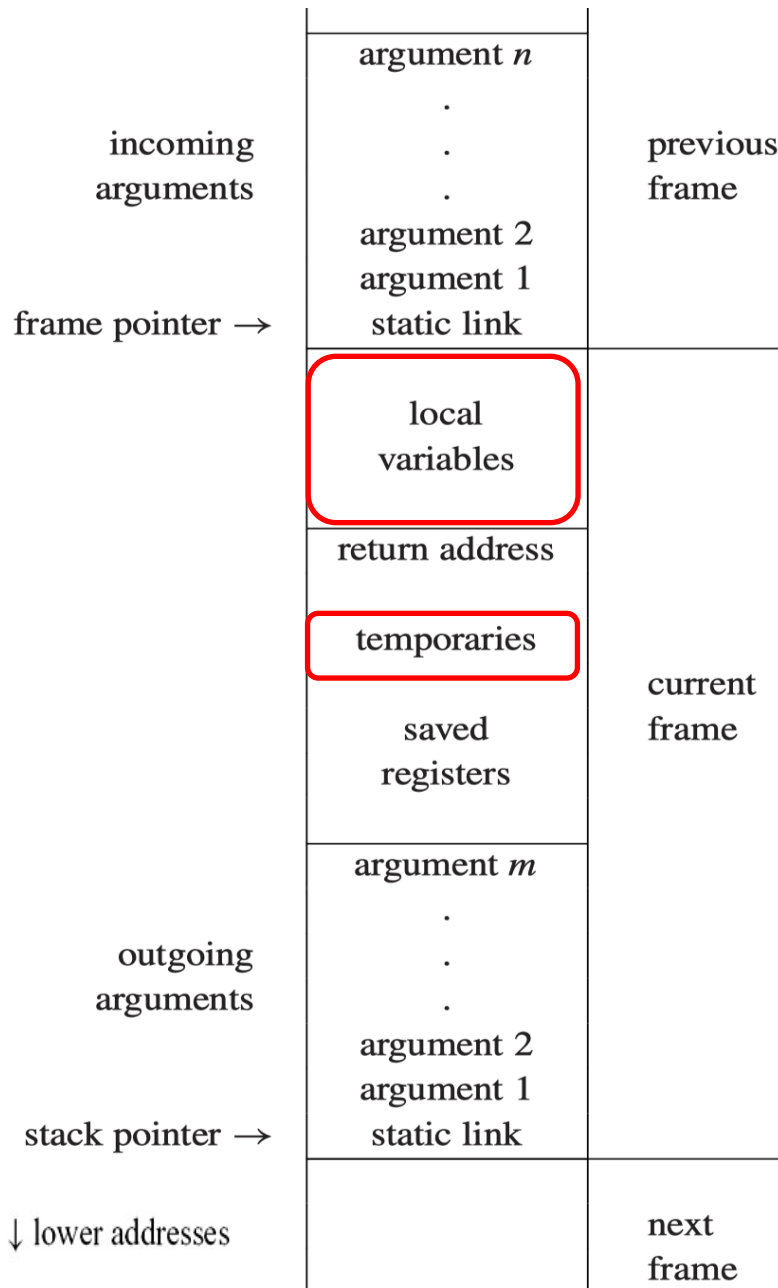
- **Frame point**为特定寄存器(如rbp, SP), 其值为栈上的内存地址
- 该地址内存中所保存值是stack link (某个函数的frame point)

A Typical Stack Frame: Return Address



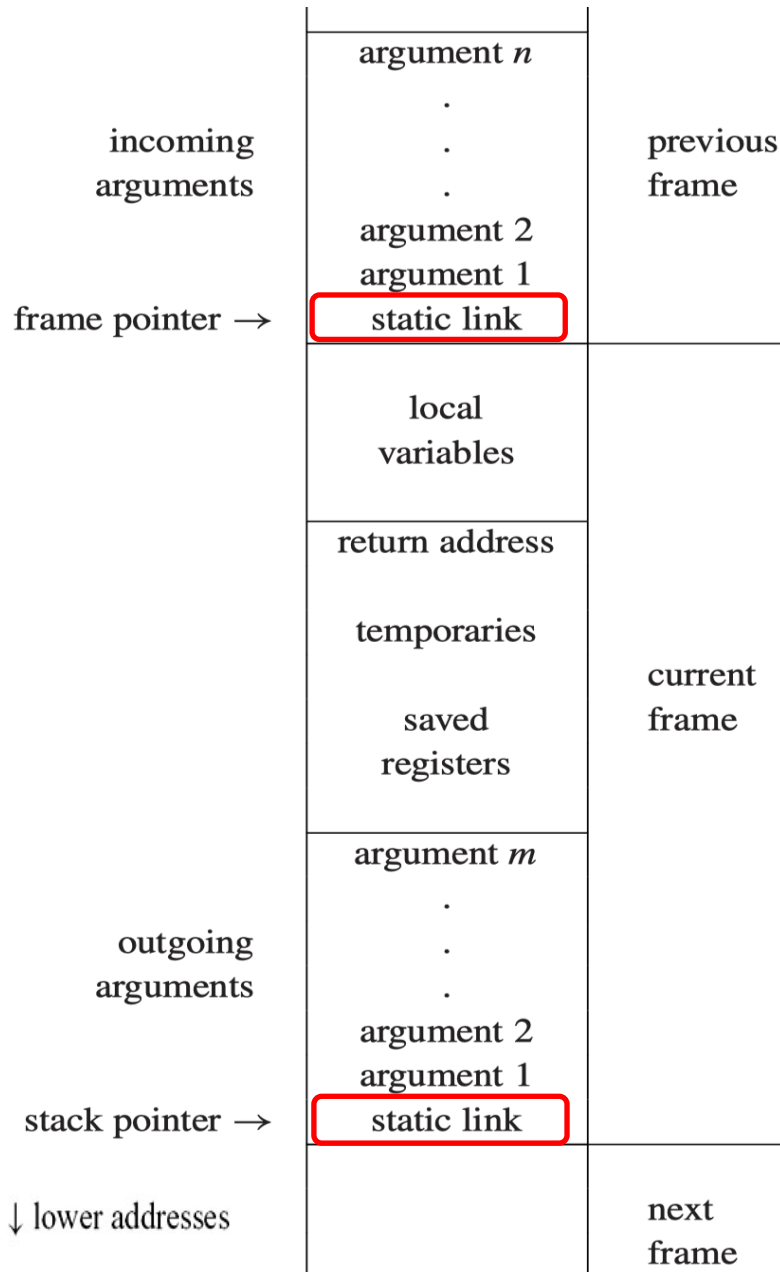
- **Return address:** where (within the calling function) control should return.
 - created by the CALL instruction

A Typical Stack Frame: Locals and Temporaries



- Some of the **local variables** and **temporaries** are in this frame, other local variables are kept in registers

A Typical Stack Frame: Static Link



NOTE: Static links may skip dynamic frames between f and g .

They always point to the most recent frame of the routine that **statically encloses** the current routine.

Limitation of Stack Frame

- **Hard to support higher-order function:** The combination of *nested functions* and *functions as arguments & returns*.

	Pascal , Tiger	C	ML, LISP, Haskell
Nested functions	√	×	√
Procedure passed as arguments and results	×	√	√

- In languages supporting *high-order functions*, it may be necessary to keep local variables after a function has returned
- But until know, we assume local variables will not be used after *f* returns (so we use the *stack*)!

Summary

- Activation records (stack frames) manage function variables and state
- Stack frames contain parameters, local variables, return address, and more
- Variables can be allocated in registers or in memory
- Static links and display provide access to variables in outer scopes



Thank you all for your attention

Slides from an SJUT Course about Static Links and Display

Lexical Scope with Nested Procedures

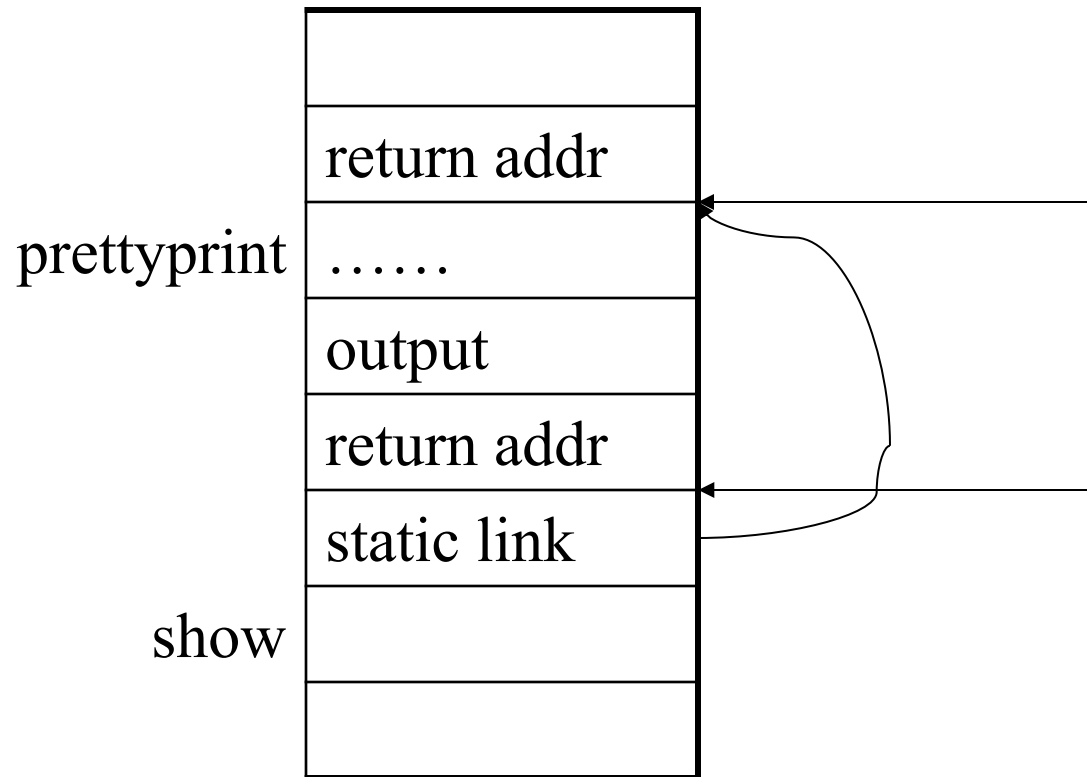
```
1.  type tree = {key:string, left: tree, right: tree}
2.
3.  function prettyprint(tree: tree): string =
4.      let
5.          var output := ""
6.
7.          function write(s:string) =
8.              output := concat(output, s)
9.
10.         function show(n:int, t:tree) =
11.             let function indent(s:string) =
12.                 (for(i:=1 to n)
13.                  do write(" ");
14.                  output := concat(output,s);write("\n"))
15.             in if t=nil then indent(".")
16.                else (indent(t.key);
17.                     show(n+1, t.left);
18.                     show(n+1, t.right))
19.         end
20.
21.     in show(0, tree); output
22. end
```

Lexical Scope with Nested Procedures

main	1
prettyprint	2
write	3
show	3
indent	4

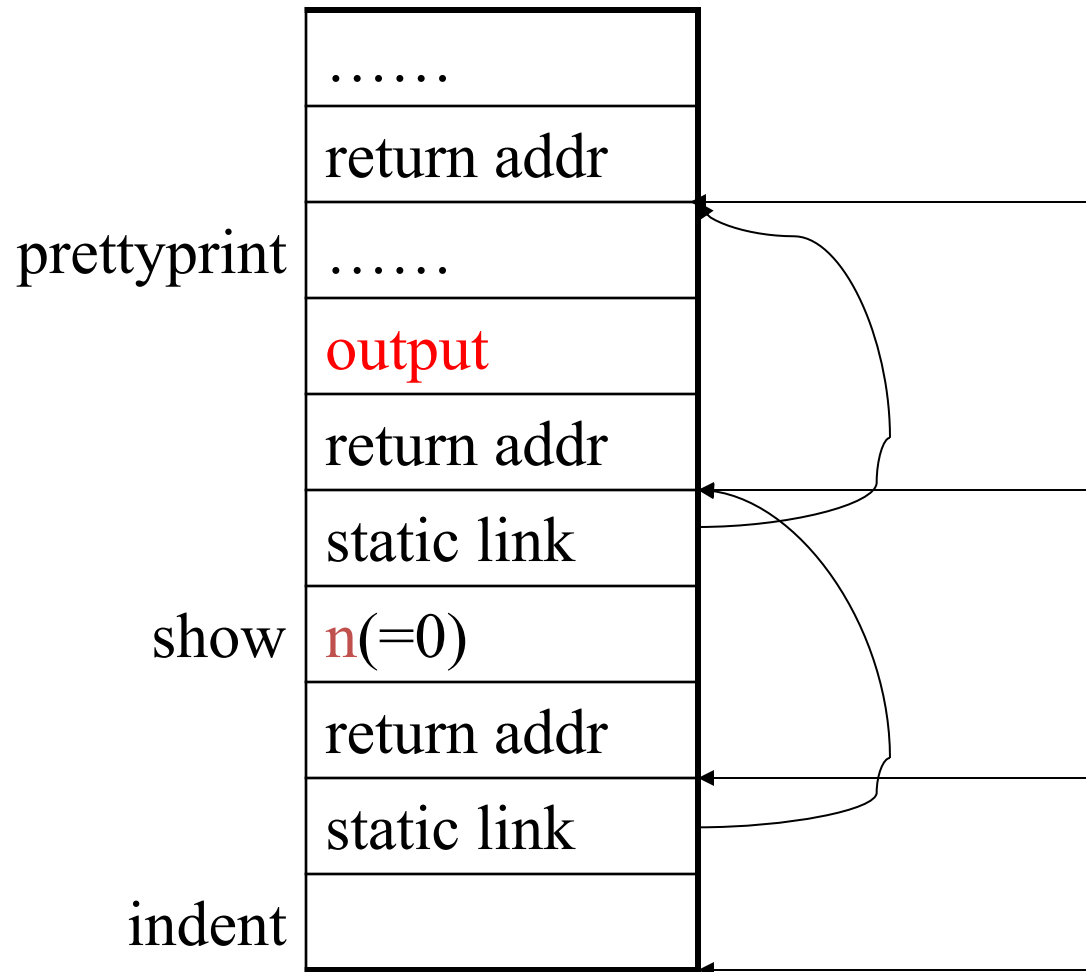
Lexical Scope with Nested Procedures

- Nesting depth
 - **Main** is at 1
 - Add 1 when going from an enclosing to an enclosed
- How can the **ident()** access **output** in **prettyprint()** and **n** in **show()**
- Using static link
 - A frame pointer passed as the first parameter to callee

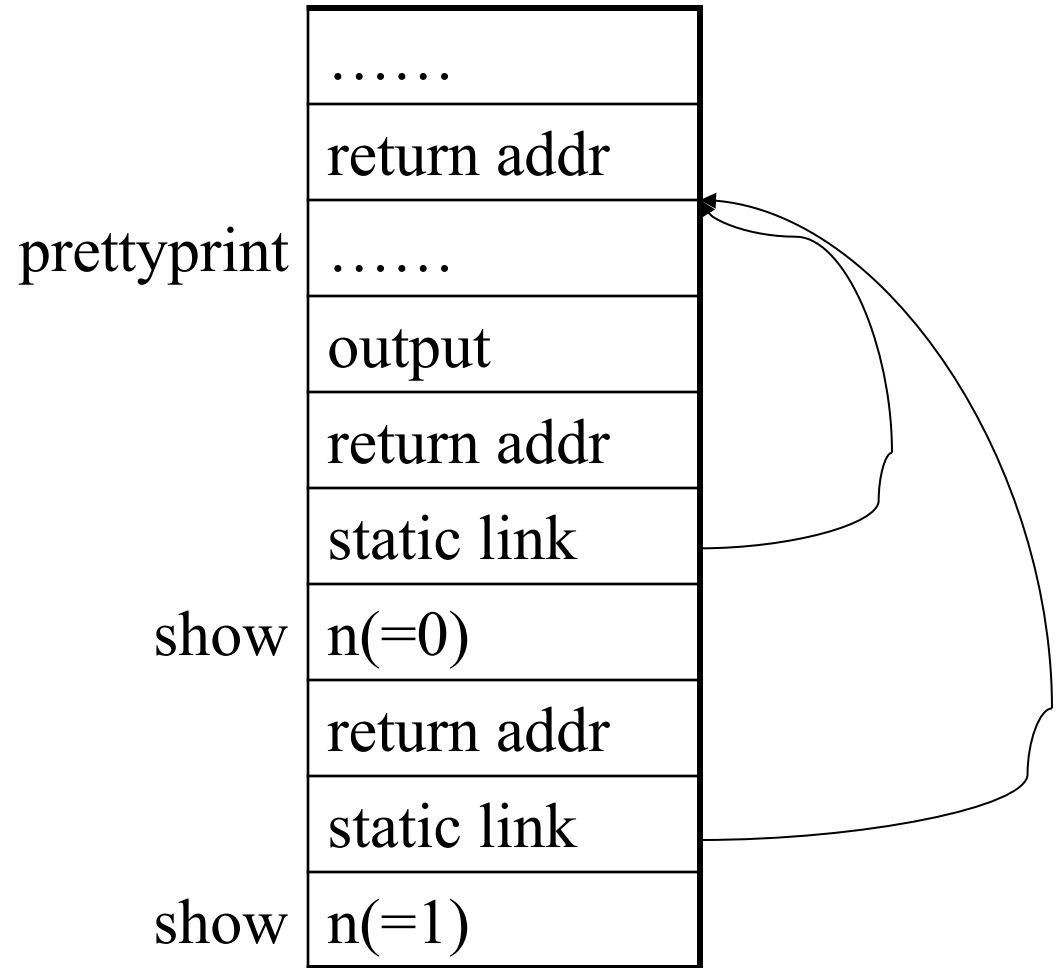


show() goes up

- one level through static link, **output** is accessible



- indent() goes up
 - one level through static link, **n** is accessible
 - two levels through static link, **output** is accessible



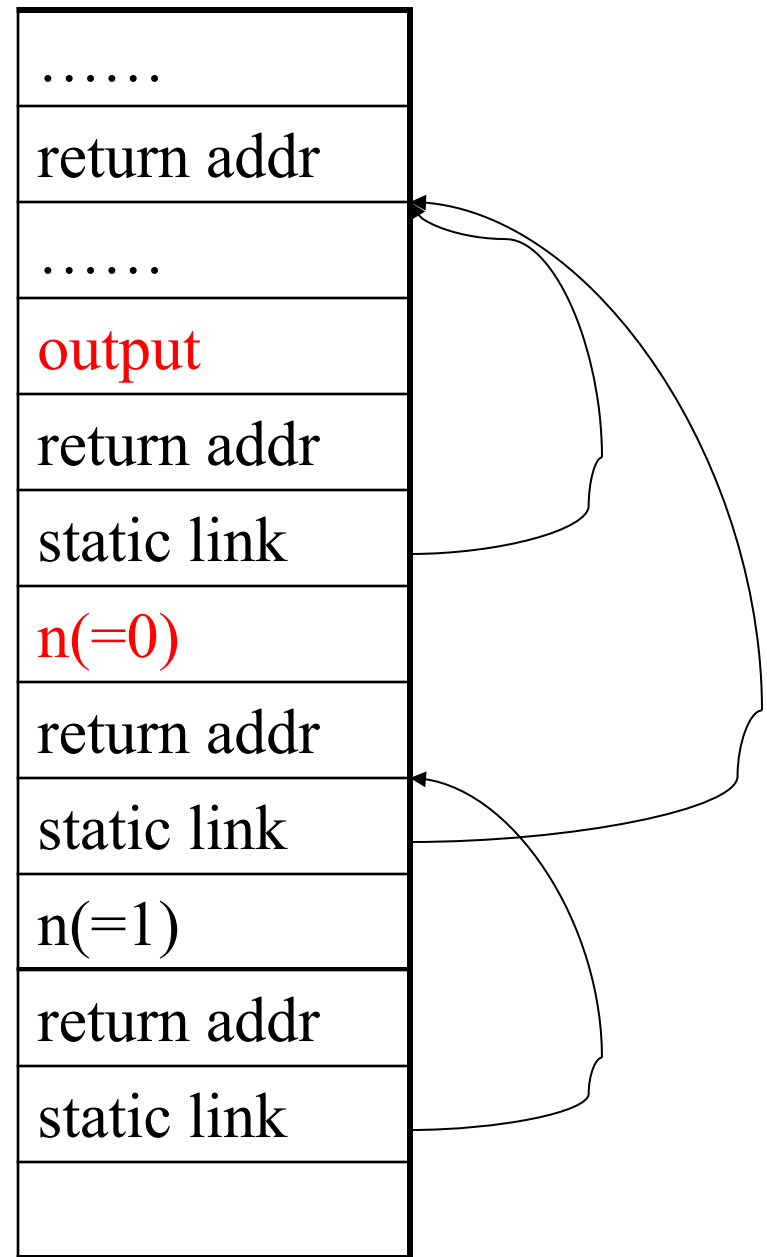
- both `show()` functions go up
 - one level through their static link, **output** is accessible

- indent()
 - Going up one level through static links, **n(=1)** show is accessible
 - Going up two levels through static link, **output** show is accessible

prettyprint

show

indent



Access Nonlocal Variables

- Procedure **p** at nesting depth n_p
- Procedure **p** refers to a nonlocal **a** with nesting depth $n_a < n_p$
 - When control is in **p**,
 - The frame for **p** is at the top of the stack
 - Follow $n_p - n_a$ static links from the frame at the top of the stack
 - After following $n_p - n_a$ links, we reach a frame for the procedure that **a** is local to.
 - $n_p - n_a$ can be computed at compile time

Set up Access Links

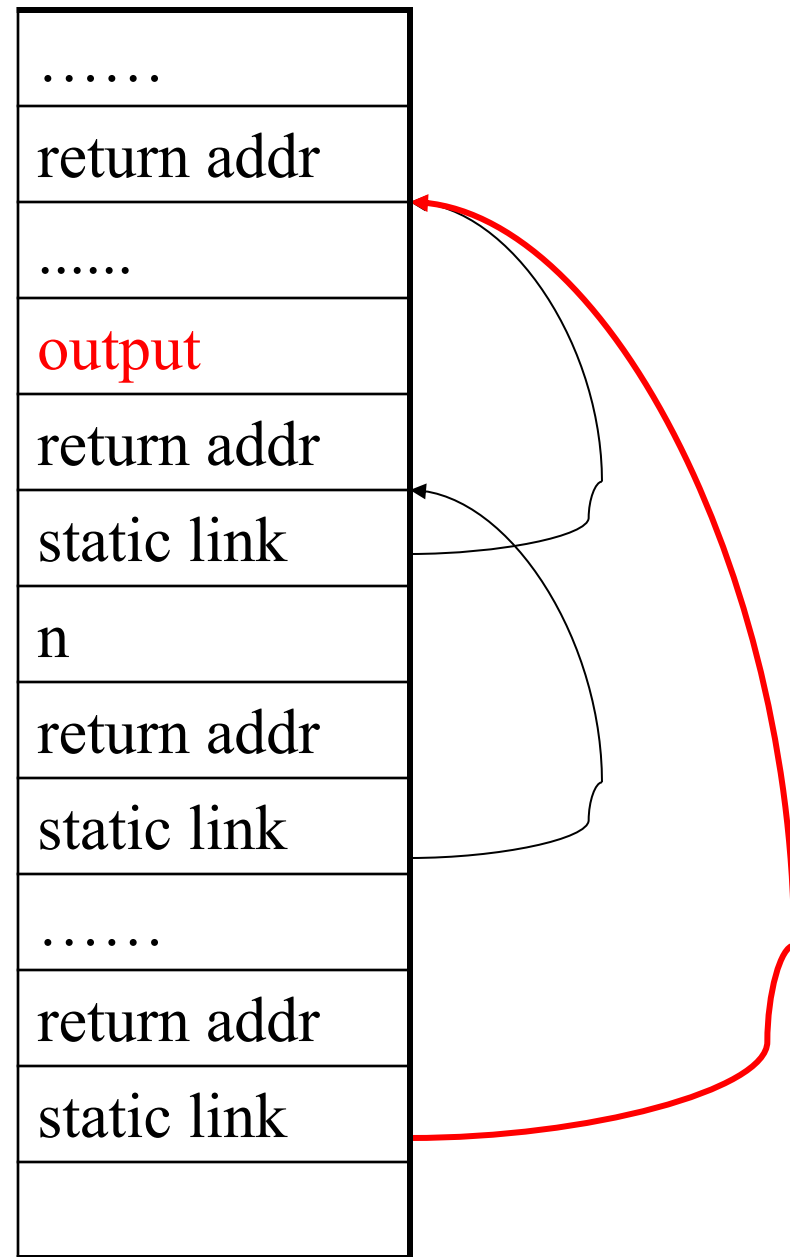
- How does the **indent()** pass the static link to **write()** ?
- **prettyprint()** is the function enclosing both **write()** and **indent()** most closely
- Go from **indent()** to **show()** , and **prettyprint()** can be achieved from **show()**

prettyprint

show

indent

write



Set up Access Links

- Procedure **p** at nesting depth n_p calls procedure **x** at nesting depth n_x .
- $n_p < n_x$
 - x must be declared within p ($n_x = n_p + 1$), static link in x must point to the frame pointer of the activation record of the caller p just below in the stack
 - Eg. **show()** calls **indent()**
 - Static link of **indent()** points to the frame pointer of **show()**

Set up Access Links

- $n_p \geq n_x$
 - The enclosing procedures at nesting depths 1, 2, ..., $n_x - 1$ of the called and calling procedures must be the same
 - Following $n_p - n_x + 1$ access links from the caller we reach the most recent activation record of the procedure that statically encloses both the called and calling procedures most closely
 - The access link is just the one to which the called procedure must point
 - $n_p - n_x + 1$ can be computed at compile time

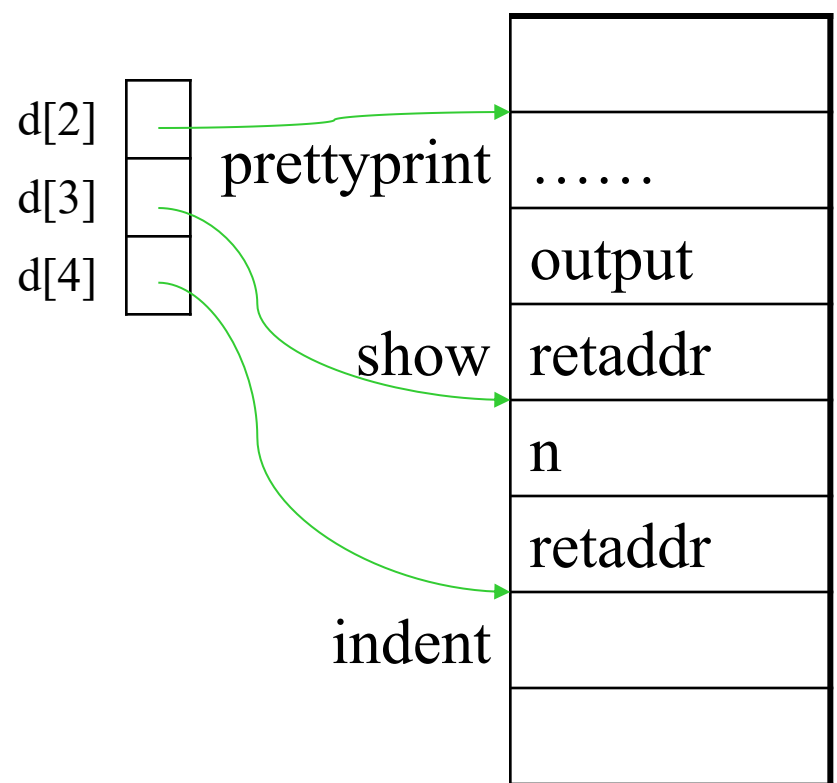
Set up Access Links

- `indent()` (at depth 4) calls `write()` (at depth 3)
 - `prettyprint()` (at nesting depth $3 - 1 = 2$) statically encloses both `write()` and `indent()` most closely
 - Following $4 - (3 - 1)$ static links from the `indent()` we reach the most recent frame of `prettyprint()`
 - The static link of `write()` just points to the frame bottom of `prettyprint()`
 - $4 - 3 + 1$ can be computed at compile time

Set up Access Links

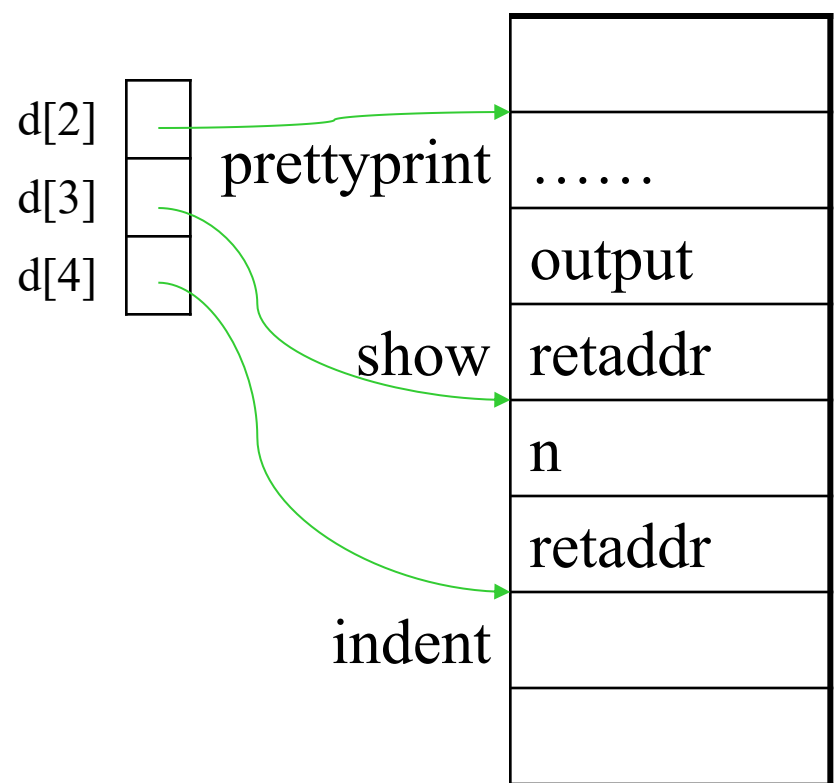
- `show()` (at depth 3) calls `show()` (at depth 3)
 - `prettyprint()` (at nesting depth 2) statically encloses both of them most closely
 - Following $3 - 3 + 1$ static links from the `show()` we reach the most recent frame of `prettyprint()`
 - The static link of `show()` just points to the frame bottom of `prettyprint()`
 - $3 - 3 + 1$ can be computed at compile time

Displays



- An array `d` of pointers to frames
- Storage for a nonlocal `a` at nesting depth `i` is in the frame pointed by display element `d[i]`

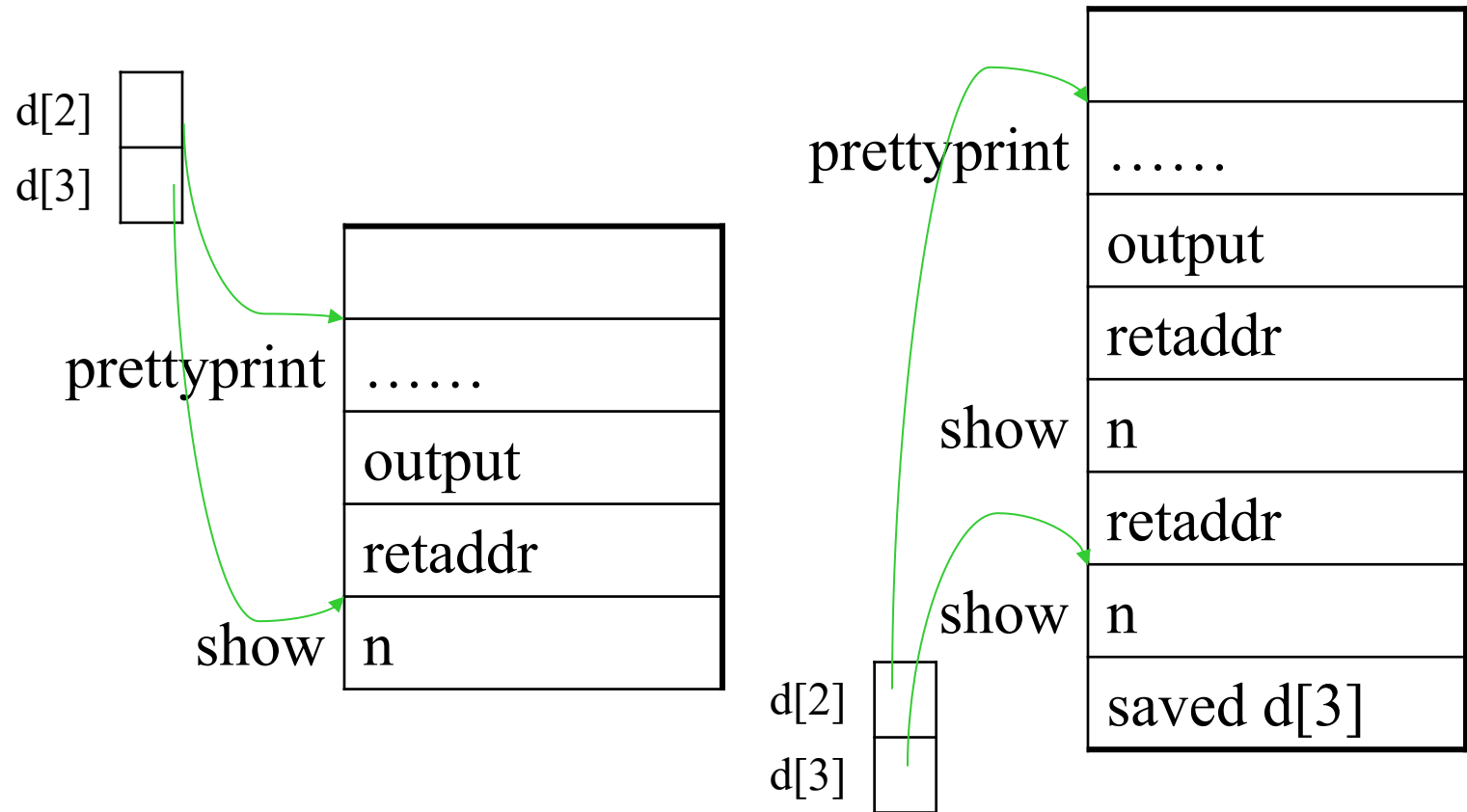
- Displays



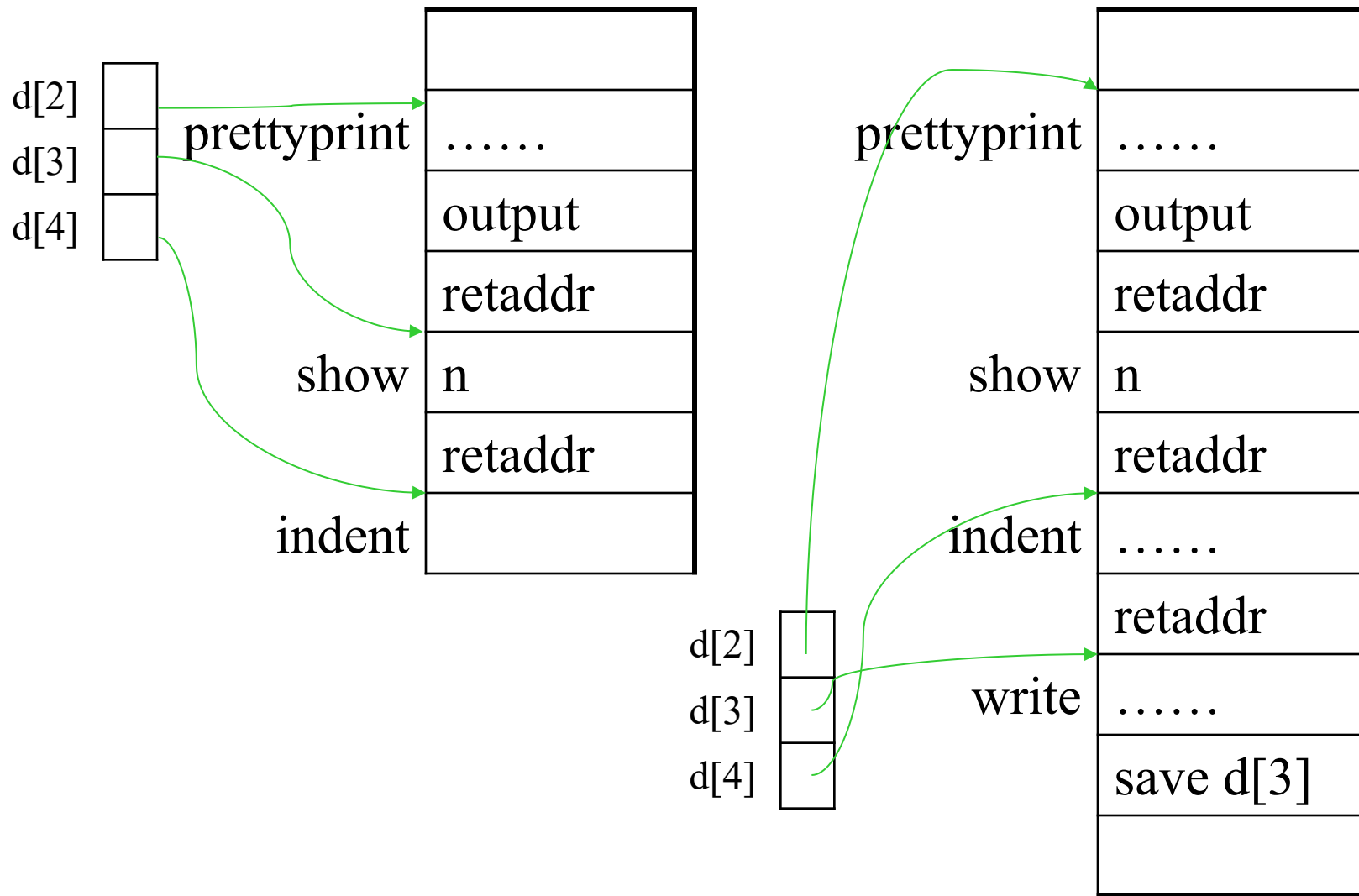
- If control is in an activation of p at nesting depth j
 - The first $j-1$ elements of the display point to the most recent activations of the procedures that lexically enclose procedure p
 - $d[j]$ points to the activation of p
 - Using a display is generally faster than following access links

Displays

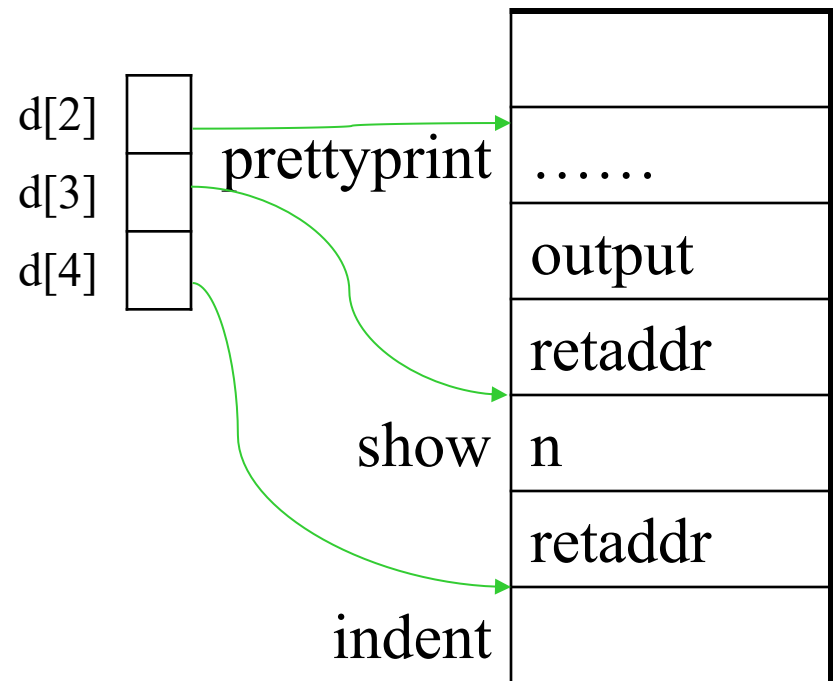
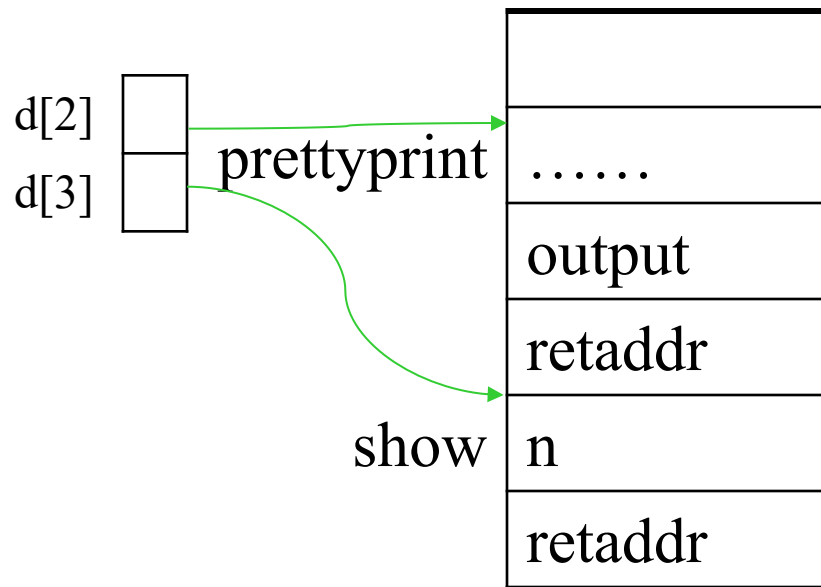
- The display changes when
 - A new activation occurs
 - Control returns from the new activation
- Simple arrangement
 - Uses static links
 - The display is updated by following the chain of static links



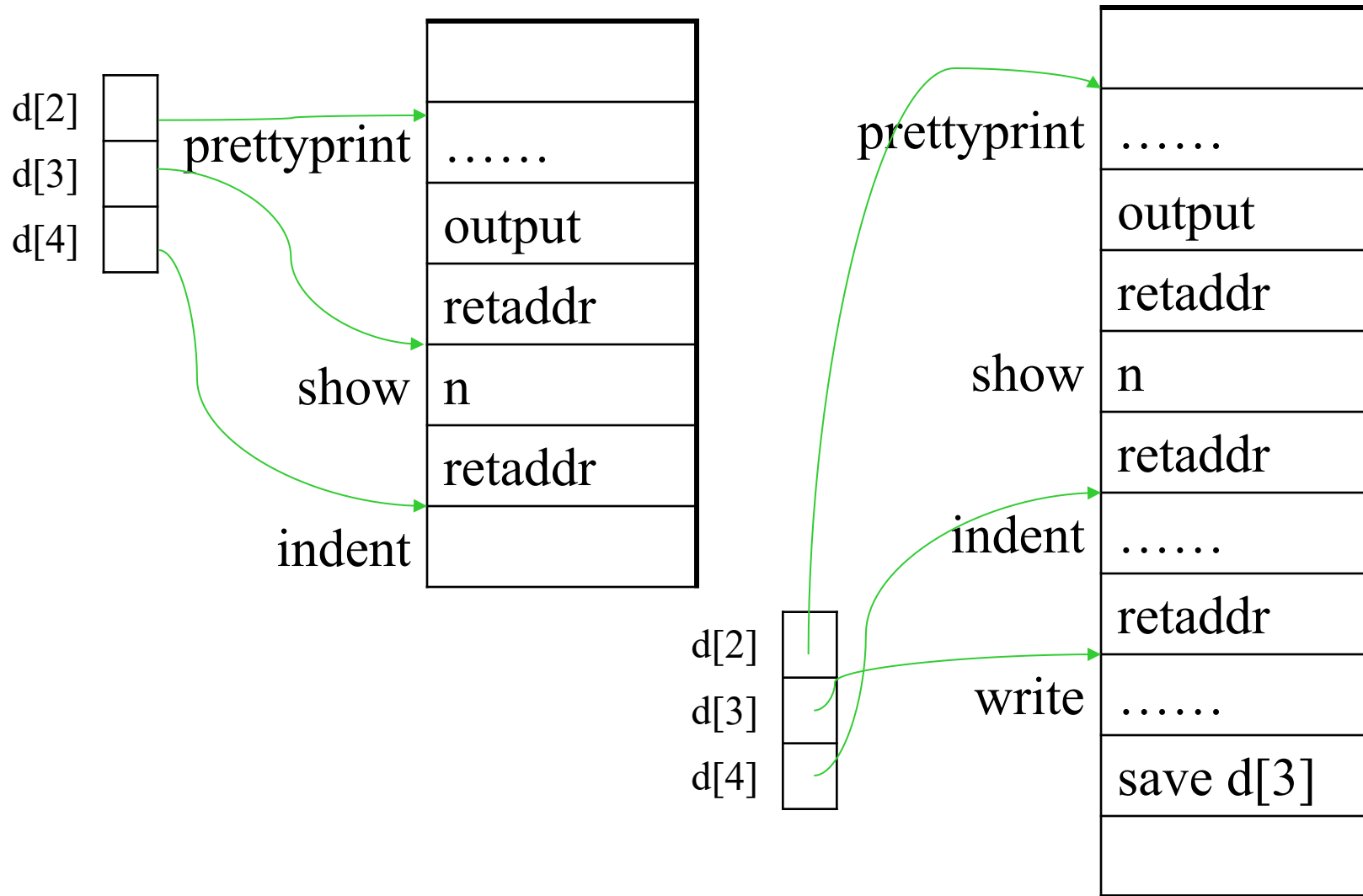
- When a new activation record for a procedure at nesting depth i is set up
 - Save the value of $d[i]$ in the new activation record
 - Set $d[i]$ to point to the new activation record



- Just before an activation ends
 - `d[i]` is reset to the saved value



- Show() at nesting depth j calls indent at depth i , $j < i$,
 - $i=j+1$
 - The first j elements of display do not need to be changed
 - We set $d[i]$ to the new frame



- indent() at nesting depth 4 calls write() at depth 3
 - Save the old value of d[3] in the new frame
 - Make d[3] points to the new frame of write()

Lambda Lifting

- When g calls f , each variable of g that is actually accessed by f (or by any function nested inside f) is passed to f as an extra argument.